

SymPy Bug Report

1 Bug 18: sinh Falsely Evaluates a Nonzero Split Imaginary-Period Argument to Zero

1.1 Minimal Reproducer

```
from sympy import I, pi, simplify, sinh

expr = sinh(I*pi + I*(pi - 1))
combined = sinh(I*(2*pi - 1))

print("sinh(I*pi + I*(pi - 1)) =", expr)
print("sinh(I*(2*pi - 1)) =", combined)
print("difference =", simplify(expr - combined))
```

1.2 Observed Behavior

```
sinh(I*pi + I*(pi - 1)) = 0
sinh(I*(2*pi - 1)) = -I*sin(1)
difference = I*sin(1)
cmath expected = -0.8414709848078966j
```

SymPy evaluates the split argument to zero even though the algebraically combined argument evaluates to $-I \sin(1)$. The recorded numerical oracle is also nonzero, so this is not merely a different but equivalent symbolic form.

1.3 Expected Behavior

The expression $\sinh(I\pi + I(\pi - 1))$ should evaluate to $-I \sin(1)$, with numerical value approximately $-0.8414709848078965 I$. Equivalently, it should agree with $\sinh(I(2\pi - 1))$.

1.4 Mathematical Explanation

The two arguments are identical:

$$I\pi + I(\pi - 1) = I(2\pi - 1).$$

For real t , the identity $\sinh(It) = I \sin(t)$ gives

$$\sinh(I(2\pi - 1)) = I \sin(2\pi - 1) = -I \sin(1).$$

Since $\sin(1) \neq 0$, the correct value is nonzero. The function $\sinh$ is entire, so there is no branch convention or missing integration constant that could make the split expression evaluate to zero while the combined expression evaluates to $-I \sin(1)$.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant source locations as follows.

File	Relevant function or helper
<code>sympy/functions/elementary/hyperbolic.py</code>	<code>sinh.eval</code> , lines 189–208
<code>sympy/functions/elementary/trigonometric.py</code>	<code>sin.eval</code> , lines 301–382
<code>sympy/functions/elementary/trigonometric.py</code>	<code>_peeloff_pi</code> , lines 154–159

The traced call path starts in `sinh.eval`. SymPy extracts the coefficient of I with `_imaginary_unit_as_coefficient` and rewrites the call as $I \sin(i_coeff)$. For the unevaluated split argument, that coefficient is the addend structure $(-1 + \pi) + \pi$. Then `sin.eval` calls `_peeloff_pi`.

The source-level issue is in `_peeloff_pi`. The helper iterates over `Add.make_args(arg)` and uses `a.coeff(pi)` to remove rational multiples of π . For the nested addend $-1 + \pi$, `a.coeff(pi)` is 1. The diagnosis reports that this increments the peeled π -coefficient but fails to preserve the non- π remainder -1 in the residual terms.

As a result, the argument $\pi + (\pi - 1)$ is treated as the exact multiple 2π . The downstream trigonometric simplification evaluates $\sin(2\pi)$ to zero, and the hyperbolic rewrite propagates this as $\sinh(I\pi + I(\pi - 1)) = I \cdot 0 = 0$. A plausible fix direction supported by the diagnosis is to peel a π multiple from an addend only when the whole addend is a pure rational multiple of π , or otherwise subtract the peeled multiple and keep any nonzero remainder in the residual terms.

1.6 Additional Failing Instances

The independent verification checks the family

$$\sinh(I\pi + I(\pi - k)), \quad k \in \{1, 2, 3, 4, 5, 6\}.$$

For every listed integer offset, the same identity applies:

$$\sinh(I\pi + I(\pi - k)) = \sinh(I(2\pi - k)) = I \sin(2\pi - k) = -I \sin(k).$$

The expected value is therefore nonzero for these integer offsets, but the dropped-remainder mechanism makes the split argument look like an exact 2π trigonometric argument.

Input / Expression	SymPy behavior	Expected behavior
$\sinh(I\pi + I(\pi - 1))$	0	$-I \sin(1)$
$\sinh(I\pi + I(\pi - 2))$	0	$-I \sin(2)$
$\sinh(I\pi + I(\pi - 3))$	0	$-I \sin(3)$
$\sinh(I\pi + I(\pi - 4))$	0	$-I \sin(4)$
$\sinh(I\pi + I(\pi - 5))$	0	$-I \sin(5)$
$\sinh(I\pi + I(\pi - 6))$	0	$-I \sin(6)$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public issues involve trigonometric periodicity and the same `_peeloff_pi` helper: SymPy issue #12328 discusses `_peeloff_pi` in a assumptions-metadata case involving $\sin(m\pi + \pi)$, while issue #17976 concerns incomplete periodic simplification such as $\sin(2n\pi + 4)$. These are related at the subsystem level but do not report the exact split imaginary-period reproducer, the wrong zero result, or the dropped non- π remainder from $\pi - 1$. The deduplication verdict is therefore a new instance of a known failure mode, and the concrete reproducer remains individually actionable as a SymPy issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import I, pi, simplify, sin, sinh

@pytest.mark.parametrize("offset", [1, 2, 3, 4, 5, 6])
def test_sinh_split_imaginary_period_preserves_remainder(offset):
    assert simplify(sinh(I*pi + I*(pi - offset)) + I*sin(offset)) == 0
```